

Mini RFC: Connection Migration

April 5, 2022

Note: This is a simplified and annotated summary of the QUIC-related IETF RFCs. It's part of a course project at Tsinghua University and not suitable for any other purposes.

We reorder paragraphs in QUIC RFCs and omit many details to improve its readability for beginners.

WARNING: DO NOT distribute this document. Modification of extracts from IETF RFCs is not granted by IETF as original authors retain copyrights of RFCs. This document is for educational purposes only.

1 Overview

The use of a connection ID allows connections to survive changes to endpoint addresses (IP address and port), such as those caused by an endpoint migrating to a new network. This section describes the process by which an endpoint migrates to a new address.

The design of QUIC relies on endpoints retaining a stable address for the duration of the handshake. An endpoint **MUST NOT** initiate connection migration before the handshake is confirmed.

If the peer sent the `disable_active_migration` transport parameter, an endpoint also **MUST NOT** send packets (including probing packets) from a different local address to the address the peer used during the handshake, unless the endpoint has acted on a `preferred_address` transport parameter from the peer. If the peer violates this requirement, the endpoint **MUST** either drop the incoming packets on that path without generating a Stateless Reset or proceed with path validation and allow the peer to migrate. Generating a Stateless Reset or closing the connection would allow third parties in the network to cause connections to close by spoofing or otherwise manipulating observed traffic.

It's OK to drop all packets from new addresses if this transport parameter is set.

Not all changes of peer address are intentional, or active, migrations. The peer could experience NAT rebinding: a change of address due to a middlebox, usually a NAT, allocating a new outgoing port or even a new outgoing IP address for a flow. **An endpoint *MUST* perform path validation if it detects any change to a peer's address, unless it has previously validated that address.**

You can perform connection migration unconditionally if path validation is not implemented.

When an endpoint has no validated path on which to send packets, it MAY discard connection state. An endpoint capable of connection migration MAY wait for a new path to become available before discarding connection state.

This document limits migration of connections to new client addresses, **except server's preferred address**. Clients are responsible for initiating all migrations. Servers do not send non-probing packets toward a client address until they see a non-probing packet from that address. If a client receives packets from an unknown server address, the client MUST discard these packets.

Optional features: *Server Preferred Address*

2 Details

2.1 Probing a New Path

An endpoint MAY probe for peer reachability from a new local address using path validation prior to migrating the connection to the new local address. Failure of path validation simply means that the new path is not usable for this connection. Failure to validate a path does not cause the connection to end unless there are no valid alternative paths available.

PATH_CHALLENGE, PATH_RESPONSE, NEW_CONNECTION_ID, and PADDING frames are **probing frames**, and all other frames are **non-probing frames**. A packet containing only probing frames is a **probing packet**, and a packet containing any other frame is a **non-probing packet**.

2.2 Initiating Connection Migration

An endpoint can migrate a connection to a new local address by sending packets containing non-probing frames from that address.

Each endpoint validates its peer's address during connection establishment. Therefore, a migrating endpoint can send to its peer knowing that the peer is willing to receive at the peer's current address. Thus, an endpoint can migrate to a new local address without first validating the peer's address.

To establish reachability on the new path, an endpoint initiates path validation on the new path. An endpoint MAY defer path validation until after a peer sends the next non-probing frame to its new address.

2.3 Responding to Connection Migration

Receiving a packet from a new peer address containing a non-probing frame indicates that the peer has migrated to that address.

If the recipient permits the migration, it MUST send subsequent packets to the new peer address and MUST initiate path validation to verify the peer's ownership of the address if validation is not already underway. If

the recipient has no unused connection IDs from the peer, it will not be able to send anything on the new path until the peer provides one.

An endpoint only changes the address to which it sends packets in response to the highest-numbered non-probing packet. This ensures that an endpoint does not send packets to an old peer address in the case that it receives reordered packets.

An endpoint MAY send data to an unvalidated peer address, but it MUST protect against potential attacks as described in §3. An endpoint MAY skip validation of a peer address if that address has been seen recently. In particular, if an endpoint returns to a previously validated path after detecting some form of spurious migration, skipping address validation and restoring loss detection and congestion state can reduce the performance impact of the attack.

After changing the address to which it sends non-probing packets, an endpoint can abandon any path validation for other addresses.

Receiving a packet from a new peer address could be the result of a NAT rebinding at the peer.

After verifying a new client address, the server SHOULD send new address validation tokens to the client.

We generally need only one path, unless we employ **Multipath QUIC**.

Related optional features: *Address Validation*

2.4 Loss Recovery & Congestion Control

When migrating, the new path might not support the endpoint's current sending rate. Therefore, the endpoint resets its congestion controller and RTT estimate. The new path might not have the same ECN capability. Therefore, the endpoint validates ECN capability as described.

The capacity available on the new path might not be the same as the old path. Packets sent on the old path MUST NOT contribute to congestion control or RTT estimation for the new path.

On confirming a peer's ownership of its new address, an endpoint MUST immediately reset the congestion controller and round-trip time estimator for the new path to initial values unless the only change in the peer's address is its port number. Because port-only changes are commonly the result of NAT rebinding or other middlebox activity, the endpoint MAY instead retain its congestion control state and round-trip estimate in those cases instead of reverting to initial values. In cases where congestion control state retained from an old path is used on a new path with substantially different characteristics, a sender could transmit too aggressively until the congestion controller and the RTT estimator have adapted. Generally, implementations are advised to be cautious when using previous values on a new path.

Pay attention to the order here.

There could be apparent reordering at the receiver when an endpoint sends data and probes from/to multiple addresses during the migration period, since the two resulting paths could have different round-trip times. A receiver of packets on multiple paths will still send ACK frames covering all received packets.

While multiple paths might be used during connection migration, a single congestion control context and a single loss recovery context could be adequate. For instance, an endpoint might delay switching to a new congestion control context until it is confirmed that an old path is no longer needed (such as the case described in §3.3).

A sender can make exceptions for probe packets so that their loss detection is independent and does not unduly cause the congestion controller to reduce its sending rate. An endpoint might set a separate timer when a `PATH_CHALLENGE` is sent, which is canceled if the corresponding `PATH_RESPONSE` is received. If the timer fires before the `PATH_RESPONSE` is received, the endpoint might send a new `PATH_CHALLENGE` and restart the timer for a longer period of time. This timer **SHOULD** be set as described in PTO computation and **MUST NOT** be more aggressive.

3 Security Consideration

3.1 Peer Address Spoofing

It is possible that a peer is spoofing its source address to cause an endpoint to send excessive amounts of data to an unwilling host. If the endpoint sends significantly more data than the spoofing peer, connection migration might be used to amplify the volume of data that an attacker can generate toward a victim.

An endpoint is required to validate a peer's new address to confirm the peer's possession of the new address. Until a peer's address is deemed valid, an endpoint limits the amount of data it sends to that address. In the absence of this limit, an endpoint risks being used for a denial-of-service attack against an unsuspecting victim.

If an endpoint skips validation of a peer address as described above, it does not need to limit its sending rate.

3.2 On-Path Address Spoofing

An on-path attacker could cause a spurious connection migration by copying and forwarding a packet with a spoofed address such that it arrives before the original packet. The packet with the spoofed address will be seen to come from a migrating connection, and the original packet will be seen as a duplicate and dropped. After a spurious migration, validation of the source address will fail because the entity at the source address does not have the necessary cryptographic keys to read or respond to the `PATH_CHALLENGE` frame that is sent to it even if it wanted to.

To protect the connection from failing due to such a spurious migration, an endpoint **MUST** revert to using the last validated peer address when validation of a new peer address fails. Additionally, receipt of packets with higher packet numbers from the legitimate peer address will trigger another connection migration. This will cause the validation of the address of the

spurious migration to be abandoned, thus containing migrations initiated by the attacker injecting a single packet.

If an endpoint has no state about the last validated peer address, it **MUST** close the connection silently by discarding all connection state. This results in new packets on the connection being handled generically. **For instance, an endpoint MAY send a Stateless Reset in response to any further incoming packets.**

3.3 Off-Path Packet Forwarding

An off-path attacker that can observe packets might forward copies of genuine packets to endpoints. If the copied packet arrives before the genuine packet, this will appear as a NAT rebinding. Any genuine packet will be discarded as a duplicate. If the attacker is able to continue forwarding packets, it might be able to cause migration to a path via the attacker. This places the attacker on-path, giving it the ability to observe or drop all subsequent packets.

This style of attack relies on the attacker using a path that has approximately the same characteristics as the direct path between endpoints. The attack is more reliable if relatively few packets are sent or if packet loss coincides with the attempted attack.

A non-probing packet received on the original path that increases the maximum received packet number will cause the endpoint to move back to that path. Eliciting packets on this path increases the likelihood that the attack is unsuccessful. Therefore, mitigation of this attack relies on triggering the exchange of packets.

In response to an apparent migration, endpoints **MUST** validate the previously active path using a `PATH_CHALLENGE` frame. This induces the sending of new packets on that path. If the path is no longer viable, the validation attempt will time out and fail; if the path is viable but no longer desired, the validation will succeed but only results in probing packets being sent on the path.

An endpoint that receives a `PATH_CHALLENGE` on an active path **SHOULD** send a non-probing packet in response. If the non-probing packet arrives before any copy made by an attacker, this results in the connection being migrated back to the original path. Any subsequent migration to another path restarts this entire process.

This defense is imperfect, but this is not considered a serious problem. If the path via the attack is reliably faster than the original path despite multiple attempts to use that original path, it is not possible to distinguish between an attack and an improvement in routing.

An endpoint could also use heuristics to improve detection of this style of attack. For instance, NAT rebinding is improbable if packets were recently received on the old path; similarly, rebinding is rare on IPv6 paths. Endpoints can also look for duplicated packets. Conversely, a change in connection ID is more likely to indicate an intentional migration rather than

an attack.

3.4 Privacy

Using a stable connection ID on multiple network paths would allow a passive observer to correlate activity between those paths. An endpoint that moves between networks might not wish to have their activity correlated by any entity other than their peer, so different connection IDs are used when sending from different local addresses. For this to be effective, endpoints need to ensure that connection IDs they provide cannot be linked by any other entity.

At any time, endpoints MAY change the Destination Connection ID they transmit with to a value that has not been used on another path.

An endpoint MUST NOT reuse a connection ID when sending from more than one local address – for example, when initiating connection migration as described in §2.2 or when probing a new network path as described in §2.1.

Similarly, an endpoint MUST NOT reuse a connection ID when sending to more than one destination address. Due to network changes outside the control of its peer, an endpoint might receive packets from a new source address with the same Destination Connection ID field value, in which case it MAY continue to use the current connection ID with the new remote address while still sending from the same local address.

These requirements regarding connection ID reuse apply only to the sending of packets, as unintentional changes in path without a change in connection ID are possible. For example, after a period of network inactivity, NAT rebinding might cause packets to be sent on a new path when the client resumes sending. An endpoint responds to such an event as described in §2.3.

Using different connection IDs for packets sent in both directions on each new network path eliminates the use of the connection ID for linking packets from the same connection across different network paths. Header protection ensures that packet numbers cannot be used to correlate activity. This does not prevent other properties of packets, such as timing and size, from being used to correlate activity.

An endpoint SHOULD NOT initiate migration with a peer that has requested a zero-length connection ID, because traffic over the new path might be trivially linkable to traffic over the old one. If the server is able to associate packets with a zero-length connection ID to the right connection, it means that the server is using other information to demultiplex packets. For example, a server might provide a unique address to every client – for instance, using HTTP alternative services [RFC 7838]. Information that might allow correct routing of packets across multiple network paths will also allow activity on those paths to be linked by entities other than the peer.

A client might wish to reduce linkability by switching to a new connec-

tion ID, source UDP port, or IP address (see [RFC8981]) when sending traffic after a period of inactivity. Changing the address from which it sends packets at the same time might cause the server to detect a connection migration. This ensures that the mechanisms that support migration are exercised even for clients that do not experience NAT rebindings or genuine migrations. Changing address can cause a peer to reset its congestion control state (see §2.4), so addresses SHOULD only be changed infrequently.

An endpoint that exhausts available connection IDs cannot probe new paths or initiate migration, nor can it respond to probes or attempts by its peer to migrate. To ensure that migration is possible and packets sent on different paths cannot be correlated, endpoints SHOULD provide new connection IDs before peers migrate. If a peer might have exhausted available connection IDs, a migrating endpoint could include a `NEW_CONNECTION_ID` frame in all packets sent on a new network path.

4 Server's Preferred Address

QUIC allows servers to accept connections on one IP address and attempt to transfer these connections to a more preferred address shortly after the handshake. This is particularly useful when clients initially connect to an address shared by multiple servers but would prefer to use a unicast address to ensure connection stability. This section describes the protocol for migrating a connection to a preferred server address.

Migrating a connection to a new server address mid-connection is not supported by the version of QUIC specified in this document. If a client receives packets from a new server address when the client has not initiated a migration to that address, the client SHOULD discard these packets.

You have to implement *QUIC Handshake and Connection Migration* before implementing this feature.

4.1 Communicating a Preferred Address

A server conveys a preferred address by including the `preferred_address` transport parameter in the TLS handshake.

Servers MAY communicate a preferred address of each address family (IPv4 and IPv6) to allow clients to pick the one most suited to their network attachment.

Once the handshake is confirmed, the client SHOULD select one of the two addresses provided by the server and initiate path validation. A client constructs packets using any previously unused active connection ID, taken from either the `preferred_address` transport parameter or a `NEW_CONNECTION_ID` frame.

As soon as path validation succeeds, the client SHOULD begin sending all future packets to the new server address using the new connection ID and discontinue use of the old server address. If path validation fails, the client MUST continue sending all future packets to the server's original IP address.

4.2 Migration to a Preferred Address

A client that migrates to a preferred address **MUST** validate the address it chooses before migrating.

A server might receive a packet addressed to its preferred IP address at any time after it accepts a connection. If this packet contains a `PATH_CHALLENGE` frame, the server sends a packet containing a `PATH_RESPONSE` frame. The server **MUST** send non-probing packets from its original address until it receives a non-probing packet from the client at its preferred address and until the server has validated the new path.

The server **MUST** probe on the path toward the client from its preferred address. This helps to guard against spurious migration initiated by an attacker.

Once the server has completed its path validation and has received a non-probing packet with a new largest packet number on its preferred address, the server begins sending non-probing packets to the client exclusively from its preferred IP address. The server **SHOULD** drop newer packets for this connection that are received on the old IP address. The server **MAY** continue to process delayed packets that are received on the old IP address.

The addresses that a server provides in the `preferred_address` transport parameter are only valid for the connection in which they are provided. A client **MUST NOT** use these for other connections, including connections that are resumed from the current connection.

4.3 Interaction of Client Migration and Preferred Address

A client might need to perform a connection migration before it has migrated to the server's preferred address. In this case, the client **SHOULD** perform path validation to both the original and preferred server address from the client's new address concurrently.

If path validation of the server's preferred address succeeds, the client **MUST** abandon validation of the original address and migrate to using the server's preferred address. If path validation of the server's preferred address fails but validation of the server's original address succeeds, the client **MAY** migrate to its new address and continue sending to the server's original address.

If packets received at the server's preferred address have a different source address than observed from the client during the handshake, the server **MUST** protect against potential attacks as described in §3.1 and §3.2. In addition to intentional simultaneous migration, this might also occur because the client's access network used a different NAT binding for the server's preferred address.

Servers **SHOULD** initiate path validation to the client's new address upon receiving a probe packet from a different address.

A client that migrates to a new address SHOULD use a preferred address from the same address family for the server.

The connection ID provided in the preferred_address transport parameter is not specific to the addresses that are provided. This connection ID is provided to ensure that the client has a connection ID available for migration, but the client MAY use this connection ID on any path.

5 Application Protocol

[There is no new application protocol defined in this mini RFC.](#)